Project Assignment 1

In [40]:
```
import pandas as pd
df = pd.read_csv("/Users/justinsere/Desktop/BSAN360/Project/passing_cleaned.
```

In [41]:
```
#2
df.shape
```

Out[41]:
```
(2350, 27)
```

In [42]:
```
#3
df.head()
df.tail()
```

Out[42]:

	Unnamed: 0	Player	Tm	Age	G	GS	Cmp	Att	Cmp%	Yds	...	AY/A	Y/C
2345	2389	Kadarius Toney	KAN	24	13	2	0	1	0.0	0	...	0.0	0.0
2346	2390	Kyle Trask	TAM	25	2	0	0	1	0.0	0	...	0.0	0.0
2347	2391	Garrett Wilson	NYJ	23	17	17	0	1	0.0	0	...	0.0	0.0
2348	2392	Christian Kirk	JAX	27	12	12	1	2	50.0	-1	...	-0.5	-1.0
2349	2393	Ja'Marr Chase	CIN	23	16	16	1	1	100.0	-7	...	-7.0	-7.0

5 rows × 27 columns

In [43]:
```
#4
df.columns
```

Out[43]:
```
Index(['Unnamed: 0', 'Player', 'Tm', 'Age', 'G', 'GS', 'Cmp', 'Att', 'Cm
p%',
       'Yds', 'TD', 'TD%', 'Int', 'Int%', '1D', 'Lng', 'Y/A', 'AY/A', 'Y/
C',
       'Y/G', 'Rate', 'Sk', 'Yds-s', 'Sk%', 'NY/A', 'ANY/A', 'Year'],
      dtype='object')
```

In [44]:
```
#5
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2350 entries, 0 to 2349
Data columns (total 27 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Unnamed: 0  2350 non-null   int64
 1   Player      2350 non-null   object
 2   Tm          2350 non-null   object
 3   Age         2350 non-null   int64
 4   G           2350 non-null   int64
 5   GS          2350 non-null   int64
 6   Cmp         2350 non-null   int64
 7   Att         2350 non-null   int64
 8   Cmp%        2350 non-null   float64
 9   Yds         2350 non-null   int64
 10  TD          2350 non-null   int64
 11  TD%         2350 non-null   float64
 12  Int         2350 non-null   int64
 13  Int%        2350 non-null   float64
 14  1D          2350 non-null   int64
 15  Lng         2350 non-null   int64
 16  Y/A         2350 non-null   float64
 17  AY/A        2350 non-null   float64
 18  Y/C         2350 non-null   float64
 19  Y/G         2350 non-null   float64
 20  Rate        2350 non-null   float64
 21  Sk          2350 non-null   int64
 22  Yds-s       2350 non-null   int64
 23  Sk%         2350 non-null   float64
 24  NY/A        2350 non-null   float64
 25  ANY/A       2350 non-null   float64
 26  Year        2350 non-null   int64
dtypes: float64(11), int64(14), object(2)
memory usage: 495.8+ KB
```

Out[44]:

	Unnamed: 0	Age	G	GS	Cmp	
count	2350.000000	2350.000000	2350.000000	2350.000000	2350.000000	2350.000
mean	1196.480426	28.131489	10.342979	6.925532	106.064681	171.169
std	691.069444	4.356133	5.632477	6.412590	133.591048	210.308
min	0.000000	21.000000	1.000000	0.000000	0.000000	1.000
25%	598.250000	25.000000	5.000000	0.000000	1.000000	2.000
50%	1197.500000	27.000000	12.000000	5.000000	27.000000	48.000
75%	1793.750000	31.000000	16.000000	14.000000	207.750000	344.500
max	2393.000000	45.000000	17.000000	17.000000	490.000000	733.000

8 rows × 25 columns

In [45]:

```
#6
# I have slightly changed my questions from my project 1 submission.
# Question 1: How have average passing yards per season changed from 2001 to
```

```
# Columns: Season, Pasing Yards
# Question 2: What factors best predict quarterback performance (e.g., touch
# Columns: Passing Yards, Touchdowns, Interceptions, Attempts
# Question 3: Are modern quarterbacks more efficient than earlier-era quarte
# Columns: Season, Completion_Percentage, Passer_Rating
```

In [46]:
```
#7
project_df = df[[ "Year", "Player", "Tm", "Age", "G", "GS",
 "Cmp", "Att", "Cmp%", "Yds", "TD", "Int", "Rate"]]
```

Project Assignment 2

In [47]:
```
# Verify Missing Values
project_df.isnull().sum()
```

Out[47]:
```
Year      0
Player    0
Tm        0
Age       0
G         0
GS        0
Cmp       0
Att       0
Cmp%      0
Yds       0
TD        0
Int       0
Rate      0
dtype: int64
```

In [48]:
```
# Eliminate observations missing key data
project_df = project_df.dropna(subset=["Yds", "Att", "TD", "Int", "Rate"])
```

In [49]:
```
# Create minimum games played/started to count as a "Startin QB"
starter_df = project_df[
 (project_df["G"] >= 8) &
 (project_df["GS"] >= 6) &
 ((project_df["Yds"] >= 1500) | (project_df["Att"] >= 200))
].copy()
```

In [50]:
```
starter_df["Era"] = pd.cut(
 starter_df["Year"],
 bins=[2000, 2009, 2016, 2023],
 labels=["Early 2000s", "2010–2016", "Modern Era"])
```

In [51]:
```
# Final Clean Dataset
starter_df.to_csv("starter_qb_cleaned.csv", index=False)
```

Project Assignment 3

In [52]:
```
# Clean player names
starter_df["Player_clean"] = (
    starter_df["Player"]
    .str.strip()
    .str.replace(r"\s+", " ", regex=True)
)
```

In [53]:
```
starter_df["Tm"] = starter_df["Tm"].str.upper().str.strip()
```

In [54]:
```
# NOTE:
# The current NFL project uses a single consolidated dataset (passing_cleane
# Therefore, no dataset merging is required at this stage.
# This step was reviewed and determined to be not applicable for this projec
```

In [55]:
```
avg_yards_by_year = (
    starter_df
    .pivot_table(values="Yds", index="Year", aggfunc="mean")
    .reset_index()
)

avg_yards_by_year.head()
```

Out[55]:

	Year	Yds
0	2001	3134.290323
1	2002	3049.870968
2	2003	2783.272727
3	2004	2978.235294
4	2005	2699.914286

In [56]:
```
efficiency_by_era = (
    starter_df
    .pivot_table(values=["Rate", "Cmp%"], index="Era", aggfunc="mean")
)

efficiency_by_era
```

```
/var/folders/_k/vn7_nw850bn6cd7c6b4273dm0000gn/T/ipykernel_86372/3971415663.
py:3: FutureWarning: The default value of observed=False is deprecated and w
ill change to observed=True in a future version of pandas. Specify observed=
False to silence this warning and retain the current behavior
  .pivot_table(values=["Rate", "Cmp%"], index="Era", aggfunc="mean")
```

Out[56]:

Era	Cmp%	Rate
Early 2000s	60.590847	82.785085
2010–2016	61.765517	87.606897
Modern Era	64.575217	91.635217

In [57]:
```
qb_year_matrix = starter_df.pivot_table(
    values="Yds",
    index="Player_clean",
    columns="Year",
    aggfunc="mean"
)

qb_year_matrix.head()
```

Out[57]:

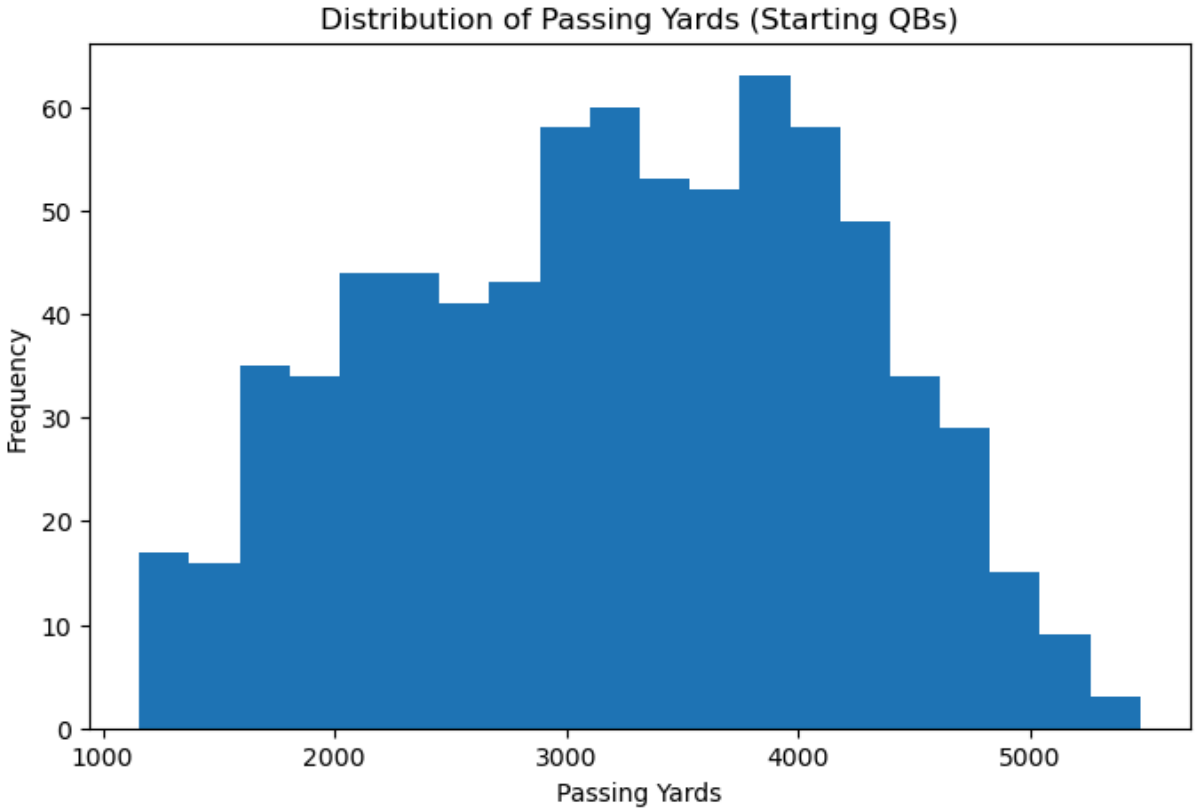
Year Player_clean	2001	2002	2003	2004	2005	2006	2007	2008	2009
A.J. Feeley	NaN	NaN	NaN	1893.0	NaN	NaN	NaN	NaN	NaN
Aaron Brooks	3832.0	3572.0	3546.0	3810.0	2882.0	NaN	NaN	NaN	NaN
Aaron Rodgers	NaN	NaN	NaN	NaN	NaN	NaN	NaN	4038.0	4434.0
Aidan O'Connell	NaN	NaN	NaN	NaN	NaN	NaN	2218.0	NaN	NaN
Alex Smith	NaN	NaN	NaN	NaN	NaN	2890.0	NaN	NaN	2350.0

5 rows × 23 columns

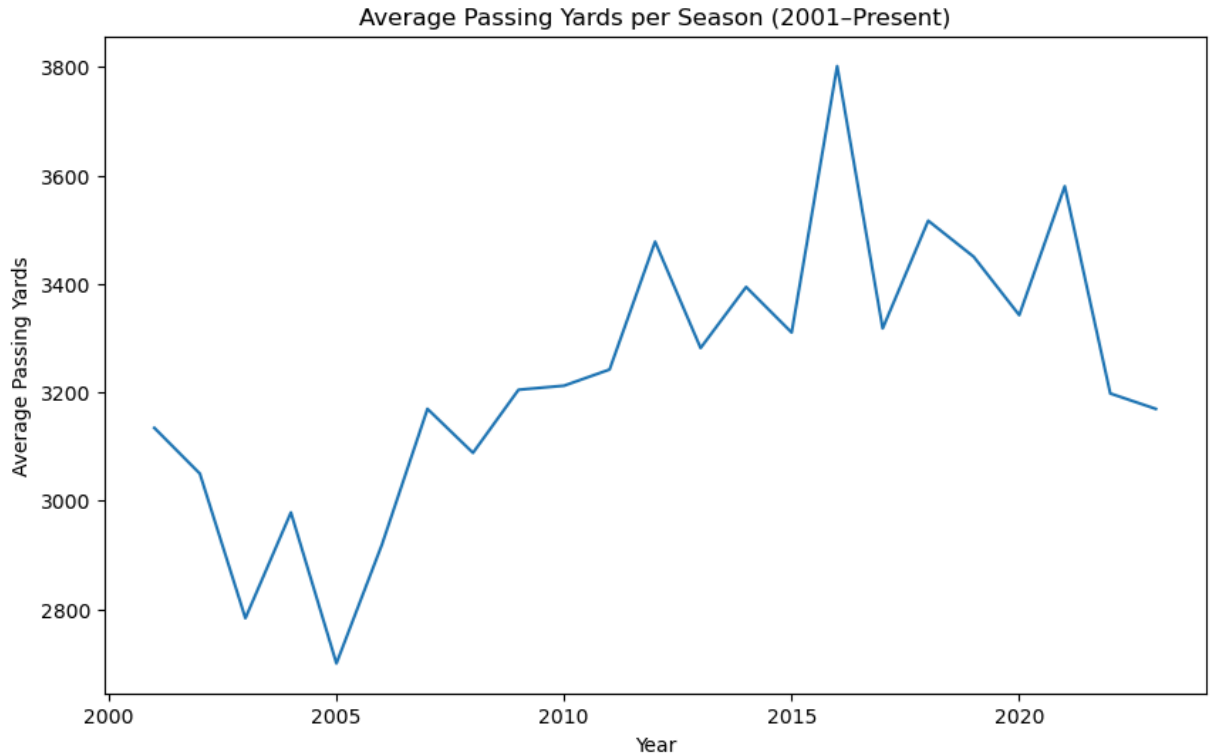
Project Assignment 4

In [58]:
```
import matplotlib.pyplot as plt
```

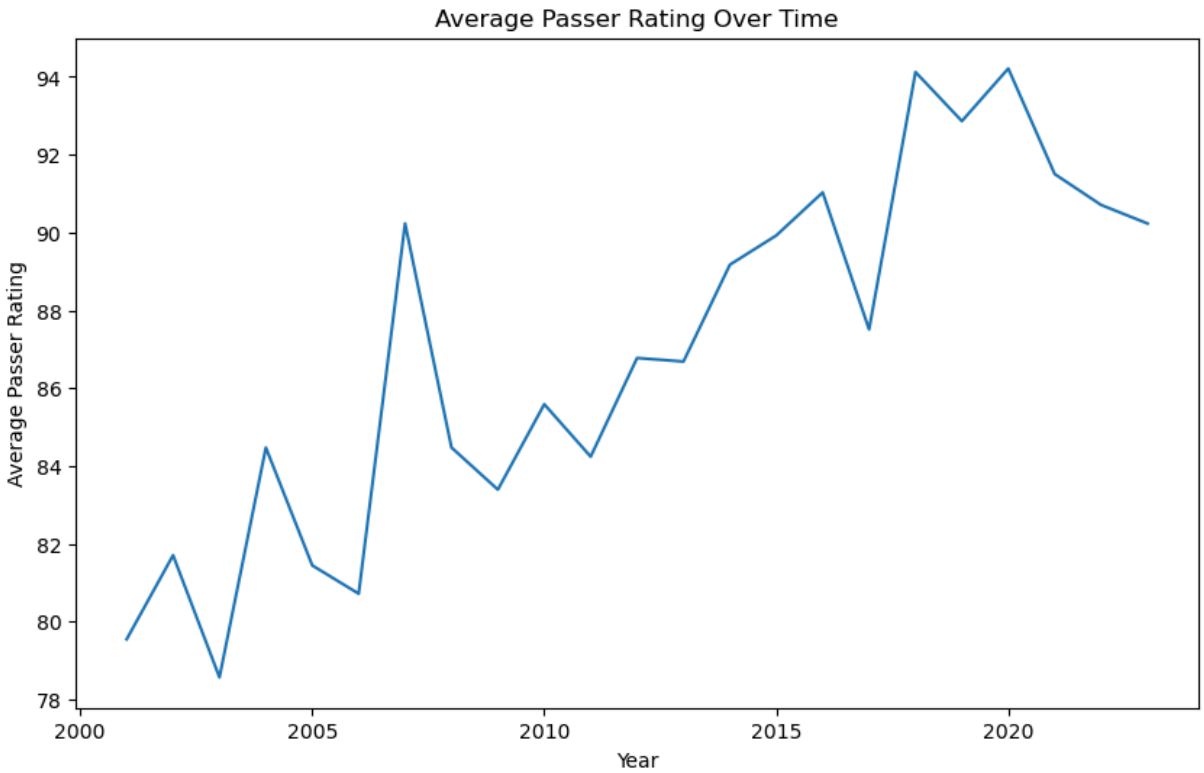
In [59]:
```
plt.figure(figsize=(8,5))
plt.hist(starter_df["Yds"], bins=20)
plt.title("Distribution of Passing Yards (Starting QBs)")
plt.xlabel("Passing Yards")
plt.ylabel("Frequency")
plt.show()
```



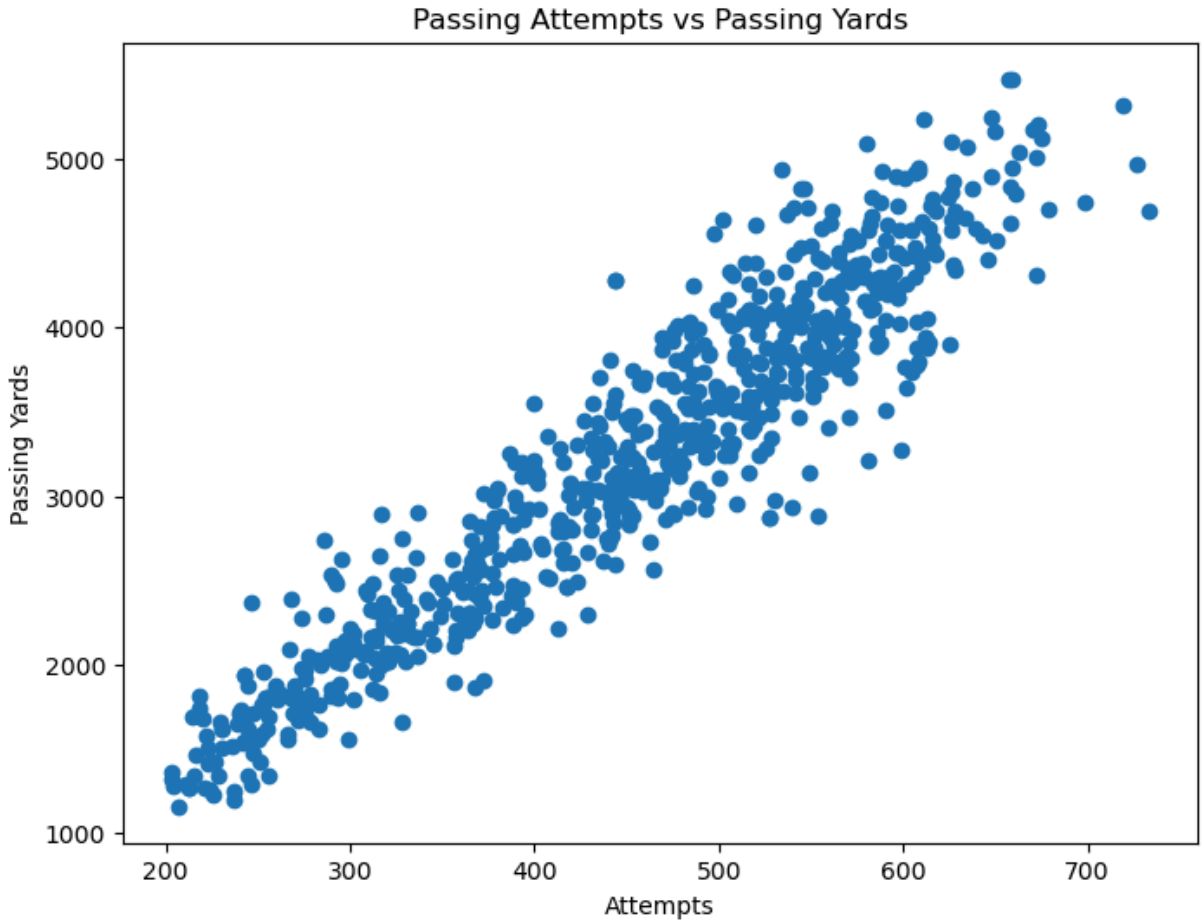
In [60]:
```
avg_yards = (
    starter_df
    .groupby("Year")["Yds"]
    .mean()
)

plt.figure(figsize=(10,6))
plt.plot(avg_yards.index, avg_yards.values)
plt.title("Average Passing Yards per Season (2001–Present)")
plt.xlabel("Year")
plt.ylabel("Average Passing Yards")
plt.show()
```



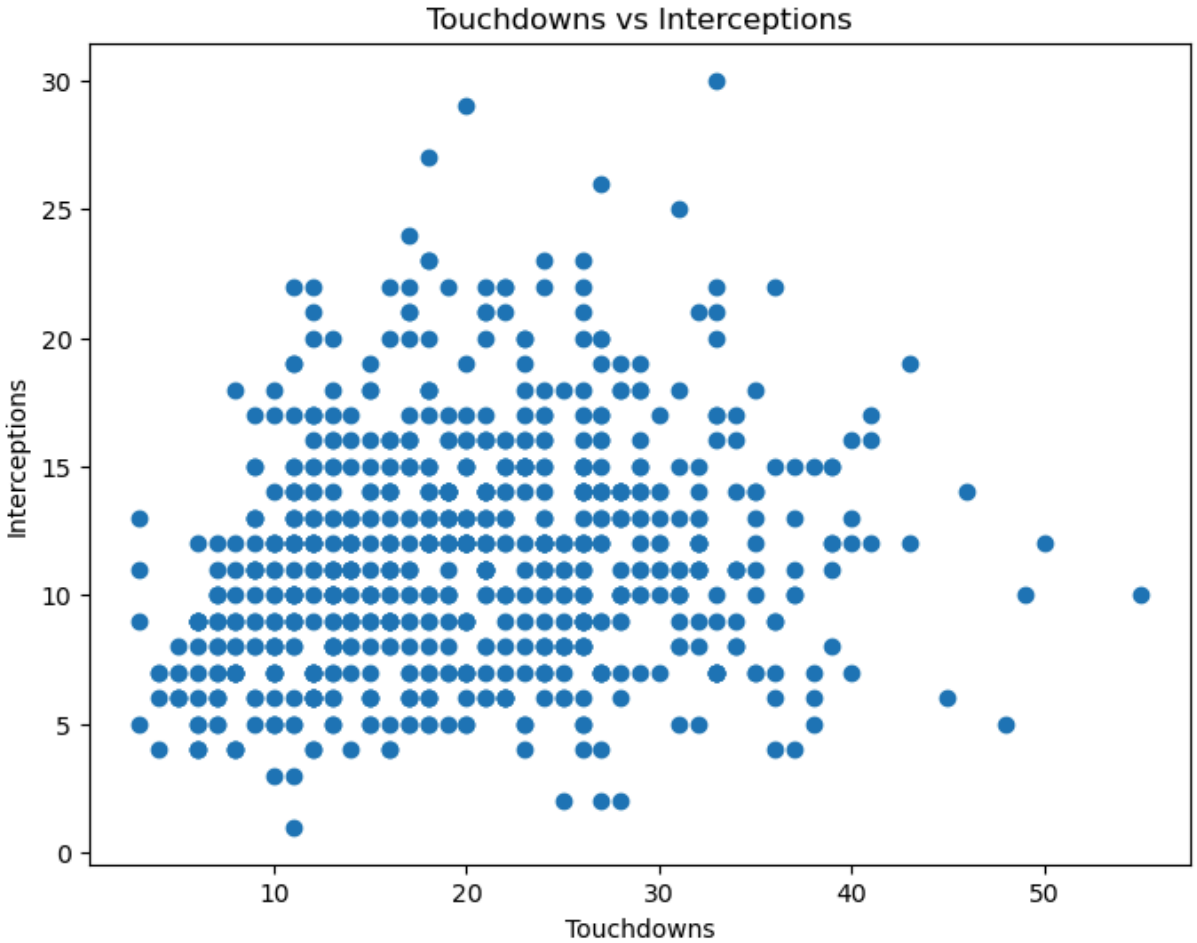
In [61]:
```
avg_rating = (
    starter_df
    .groupby("Year")["Rate"]
    .mean()
)

plt.figure(figsize=(10,6))
plt.plot(avg_rating.index, avg_rating.values)
plt.title("Average Passer Rating Over Time")
plt.xlabel("Year")
plt.ylabel("Average Passer Rating")
plt.show()
```



In [62]:
```
plt.figure(figsize=(8,6))
plt.scatter(starter_df["Att"], starter_df["Yds"])
plt.title("Passing Attempts vs Passing Yards")
plt.xlabel("Attempts")
plt.ylabel("Passing Yards")
plt.show()
```

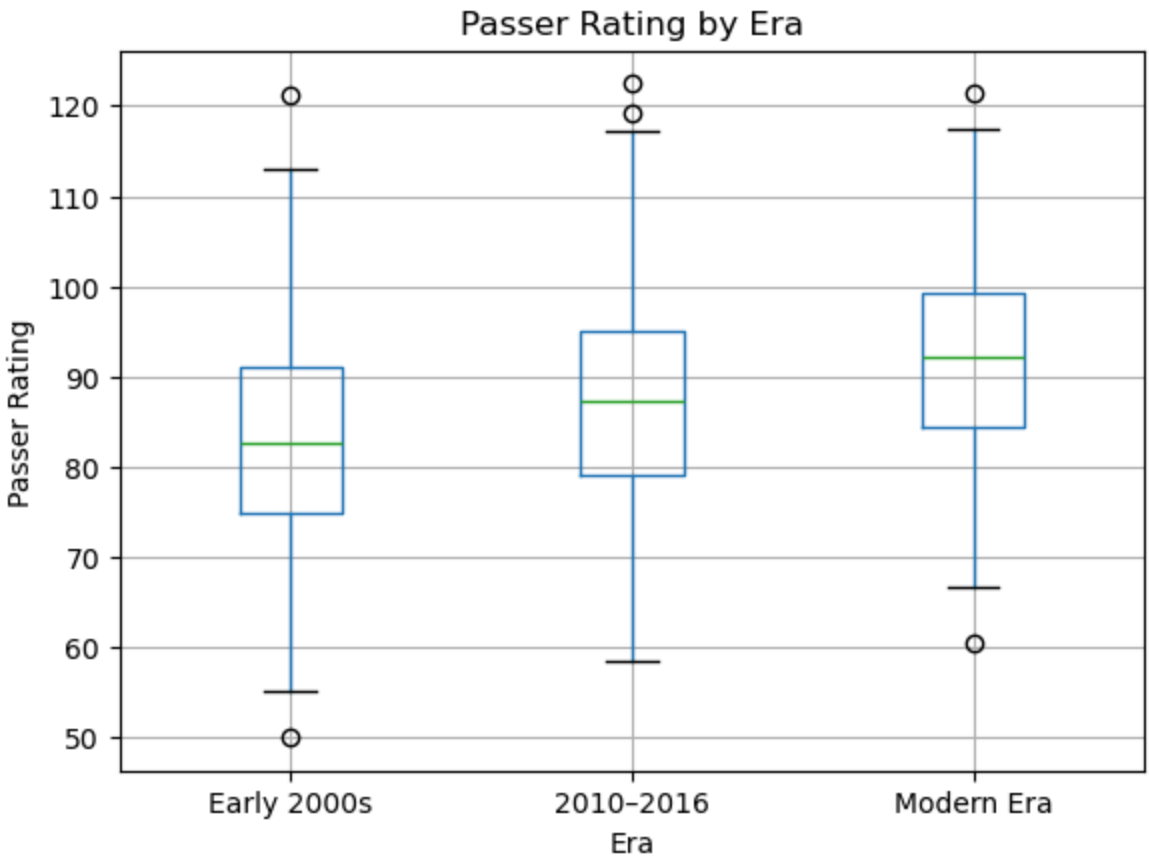


In [63]:
```
plt.figure(figsize=(8,6))
plt.scatter(starter_df["TD"], starter_df["Int"])
plt.title("Touchdowns vs Interceptions")
plt.xlabel("Touchdowns")
plt.ylabel("Interceptions")
plt.show()
```



In [64]:
```
plt.figure(figsize=(8,6))
starter_df.boxplot(column="Rate", by="Era")
plt.title("Passer Rating by Era")
plt.suptitle("")
plt.xlabel("Era")
plt.ylabel("Passer Rating")
plt.show()
```

```
<Figure size 800x600 with 0 Axes>
```



In [82]:
```
from scipy.stats import f_oneway
```

In [83]:
```
early = starter_df[starter_df["Era"] == "Early 2000s"]["Rate"]
middle = starter_df[starter_df["Era"] == "2010–2016"]["Rate"]
modern = starter_df[starter_df["Era"] == "Modern Era"]["Rate"]
```

In [84]:
```
anova_result = f_oneway(early, middle, modern)
anova_result
```

Out[84]:
```
F_onewayResult(statistic=np.float64(39.52115122003417), pvalue=np.float64
(4.75901839344233e-17))
```

Project Assignment 5

In [65]:
```
avg_by_year = (
    starter_df
    .groupby("Year")[["Yds", "TD", "Int", "Rate"]]
    .mean()
)

avg_by_year.head()
# This helps examine league-wide trends in quarterback performance across se
```

Out[65]:

Year	Yds	TD	Int	Rate
2001	3134.290323	18.064516	14.935484	79.548387
2002	3049.870968	18.451613	12.774194	81.706452
2003	2783.272727	16.727273	13.272727	78.569697
2004	2978.235294	19.235294	12.411765	84.473529
2005	2699.914286	16.057143	11.257143	81.445714

In [66]:

```
team_avg = (
    starter_df
    .groupby("Tm")[["Yds", "TD", "Rate"]]
    .mean()
    .sort_values(by="Yds", ascending=False)
)

team_avg.head()
# Grouping by team allows you to compare which franchises historically gener
```

Out[66]:

Tm	Yds	TD	Rate
LAC	4224.375000	27.125000	95.700000
NOR	4175.272727	29.181818	96.904545
GNB	3913.000000	30.000000	96.917391
SDG	3874.933333	25.733333	91.493333
LAR	3798.444444	23.333333	92.200000

In [67]:

```
top_qbs_each_year = (
    starter_df
    .sort_values(["Year", "Yds"], ascending=[True, False])
    .groupby("Year")
    .head(5)
)

top_qbs_each_year.head()
# This helps identify elite quarterback seasons and how the ceiling of passi
```

Out[67]:

	Year	Player	Tm	Age	G	GS	Cmp	Att	Cmp%	Yds	TD	Int	Rate	Era
0	2001	Kurt Warner	STL	30	16	16	375	546	68.7	4830	36	22	101.4	Early 2000s
1	2001	Peyton Manning	IND	25	16	16	343	547	62.7	4131	26	23	84.1	Early 2000s
2	2001	Brett Favre	GNB	32	16	16	314	510	61.6	3921	32	15	94.1	Early 2000s
3	2001	Aaron Brooks	NOR	25	16	16	312	558	55.9	3832	26	22	76.4	Early 2000s
4	2001	Rich Gannon	OAK	36	16	16	361	549	65.8	3828	27	9	95.5	Early 2000s

In [68]:

```
pivot_yards = starter_df.pivot_table(
    values="Yds",
    index="Year",
    aggfunc="mean"
)

pivot_yards.head()
# Shows how the league-wide passing environment changes over time.
```

Out[68]:

Year	Yds
2001	3134.290323
2002	3049.870968
2003	2783.272727
2004	2978.235294
2005	2699.914286

In [69]:

```
pivot_rating = starter_df.pivot_table(
    values="Rate",
    index="Tm",
    columns="Year",
    aggfunc="mean"
)

pivot_rating.head
# This makes it easier to identify which teams consistently have strong quar
```

Out[69]:

```
<bound method NDFrame.head of Year    2001   2002   2003    2004    2005
2006    2007    2008   2009  \
Tm
2TM      NaN    NaN    NaN     NaN     NaN    NaN   79.50     NaN    NaN
ARI    79.60   65.7   69.6   74.10   80.35   74.0   89.40   96.90   93.2
ATL    84.10   81.6   67.5   78.10   73.10   75.7   83.40   87.70   80.9
BAL    71.10   77.3   62.4   70.90   71.75   82.5  102.70   80.30   88.9
BUF    76.35   86.0   73.0   76.60   75.25   84.9   92.20   85.40   69.7
CAR    62.00   77.4   80.6   87.30   88.10   82.6   73.70   84.70   59.4
CHI    74.90   77.5   56.8     NaN   59.70   73.9   86.30   79.60   76.8
CIN    61.10   79.1   87.4   77.30  101.10   93.9   94.70   70.00   83.6
CLE    73.10   76.8   76.1   76.70   76.90   72.0     NaN   66.50   67.2
DAL      NaN   66.3   71.4   76.40   83.70   95.1  105.90   91.40   97.6
DEN    78.50   85.6   91.2   84.50   90.20   68.8   98.00   86.00   86.8
DET    76.80   59.9   63.9   77.50   72.00   79.9   97.90   72.60   61.0
GNB    94.10   85.6   90.4   92.40   70.90   72.7   96.10   93.80  103.2
HOU      NaN   62.8   69.5   83.50   77.20   82.1  100.80   92.70   98.6
IND    84.10   88.8   99.0  121.10  104.10  101.0   84.60   95.00   99.9
JAX    84.10   85.7   73.0   82.20   89.30   80.5   88.50   81.70   83.5
KAN    71.10   92.6   92.6   95.20   90.10   98.0   92.60   76.00   69.9
LAC      NaN    NaN    NaN     NaN     NaN    NaN   93.20     NaN    NaN
LAR      NaN    NaN    NaN     NaN     NaN    NaN   92.50     NaN    NaN
LVR      NaN    NaN    NaN     NaN     NaN    NaN   83.90     NaN    NaN
MIA    80.30   85.2   72.4   61.70   71.90   68.2  101.10   97.40   75.2
MIN    83.30   75.3   96.4  110.90   88.90   72.0  103.80   73.70  107.2
NOR    76.40   80.1   88.8   79.50   70.00   96.2   97.70   96.20  109.6
NWE    86.50   85.7   85.9   92.60   92.30   87.9   72.85   89.40   96.2
NYG    77.10   85.4   70.7   86.50   75.90   77.0     NaN   86.40   93.1
NYJ    75.30  104.2   82.9   91.00   72.90   82.6   77.20   81.00   63.0
OAK    95.50   97.3   64.8   74.80   77.30   55.8     NaN   77.10   50.0
PHI    84.30   86.0   79.6  104.70   70.10   95.5   89.10   86.40   92.9
PIT    81.70   85.2   75.3   98.10   98.60   75.4   81.40   80.10  100.5
SDG    72.00   76.9   67.5  104.80   89.20   92.0     NaN  105.50  104.4
SEA    70.90   87.8   88.8   83.10   98.20   76.0   92.10   87.00   75.1
SFO    94.80   85.6   80.1   70.25     NaN   74.8  113.00   80.55   81.5
STL   101.40    NaN   81.4   93.70   94.40   92.9     NaN   71.40   70.7
TAM    77.70   92.9   81.5   97.50   81.40   65.9   94.60   90.20   59.8
TEN    90.20   84.0  100.4   80.10   82.40   66.7   81.35   80.20   82.8
WAS    71.30   72.6   75.8   69.35   85.90   86.5   78.90   84.30   86.4

Year   2010  ...   2014    2015   2016   2017   2018   2019    2020    2021
\
Tm           ...
2TM     NaN  ...    NaN   69.25    NaN   74.5    NaN    NaN     NaN     NaN
ARI    65.9  ...   78.7  104.60   87.2    NaN   66.7   87.4   94.30  100.60
ATL    91.0  ...   93.9   89.00  117.1   91.4  108.1   92.1   93.30   90.40
BAL    93.6  ...   91.0   83.10   83.5   80.4   84.2  113.3   99.30   87.00
BUF    81.8  ...   87.8   99.40   89.7   89.2   67.9   85.3  107.20   92.20
CAR    58.4  ...   82.1   99.40   75.8   80.7   94.2   80.0   92.10   71.90
CHI    86.3  ...   88.6   92.30    NaN   77.5   95.4   83.0   87.15   75.05
CIN    82.4  ...   83.5  106.20   91.8   86.6   89.6   78.3   89.80  108.30
CLE    74.5  ...   76.5   86.35    NaN   60.5   93.7   78.8   95.90   83.10
DAL    88.9  ...  113.2     NaN  104.9   86.6   96.9   99.7   87.30  104.20
DEN    87.5  ...  101.5   77.15   84.6   73.3   81.2   85.1   75.40   94.90
DET    81.3  ...   85.7   97.00   93.3   99.3   89.9  106.0   96.30   91.50
GNB   101.2  ...  112.2   92.70  104.2   70.6   97.6   95.4  121.50  111.90
```

HOU	92.0	...	95.3	91.40	72.2	71.4	103.1	98.0	112.40	88.80
IND	91.9	...	96.5	84.00	96.4	81.7	98.7	88.0	97.00	94.60
JAX	90.8	...	69.5	88.20	78.8	84.7	79.8	91.2	95.90	71.90
KAN	93.0	...	93.4	95.40	91.2	104.7	113.8	105.3	108.20	98.50
LAC	NaN	...	NaN	NaN	NaN	96.0	105.5	88.5	98.30	97.70
LAR	NaN	...	NaN	NaN	76.4	100.5	101.1	86.5	90.00	102.90
LVR	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	101.40	94.00
MIA	75.4	...	92.8	88.70	93.5	80.8	92.7	85.5	91.35	90.10
MIN	69.9	...	85.2	88.70	99.3	98.3	99.7	107.4	105.00	103.10
NOR	90.9	...	97.0	101.00	101.7	103.9	115.7	116.3	106.40	NaN
NWE	111.0	...	97.4	102.20	112.2	102.8	97.7	88.0	82.90	92.50
NYG	85.3	...	92.1	93.60	86.0	80.4	92.4	87.7	80.40	84.80
NYJ	75.3	...	77.5	88.00	69.6	94.5	77.6	84.3	72.70	69.70
OAK	84.5	...	76.6	91.10	96.7	86.4	93.9	100.8	NaN	NaN
PHI	100.2	...	84.9	86.40	79.3	101.9	102.2	93.1	72.80	87.20
PIT	97.0	...	103.3	94.50	95.4	93.4	96.5	82.0	94.10	86.80
SDG	101.8	...	93.8	93.80	87.9	NaN	NaN	NaN	NaN	NaN
SEA	73.2	...	95.0	110.10	92.6	95.4	110.9	106.3	105.10	103.10
SFO	82.1	...	86.4	82.35	90.7	NaN	90.8	102.0	84.10	98.70
STL	76.5	...	84.5	69.00	NaN	NaN	NaN	NaN	NaN	NaN
TAM	95.9	...	70.5	84.20	86.1	92.2	95.3	84.3	102.20	102.10
TEN	82.2	...	NaN	91.50	95.6	79.3	92.3	117.5	106.50	89.60
WAS	77.1	...	86.9	101.60	97.2	93.9	85.7	83.7	78.50	85.90

Year	2022	2023
Tm		
2TM	79.0	79.50
ARI	87.2	89.40
ATL	88.2	83.40
BAL	91.1	102.70
BUF	96.6	92.20
CAR	NaN	73.70
CHI	85.2	86.30
CIN	100.8	94.70
CLE	88.9	NaN
DAL	91.1	105.90
DEN	84.4	98.00
DET	99.3	97.90
GNB	91.1	96.10
HOU	78.8	100.80
IND	83.9	84.60
JAX	95.2	88.50
KAN	105.2	92.60
LAC	93.2	93.20
LAR	87.4	92.50
LVR	86.3	83.90
MIA	105.5	101.10
MIN	92.5	103.80
NOR	95.2	97.70
NWE	84.8	72.85
NYG	92.5	NaN
NYJ	72.8	77.20
OAK	NaN	NaN
PHI	101.5	89.10
PIT	76.7	81.40
SDG	NaN	NaN

```
SEA   100.9   92.10
SFO   103.0  113.00
STL     NaN     NaN
TAM    90.7   94.60
TEN    94.6   81.35
WAS    84.9   78.90

[36 rows x 23 columns]>
```

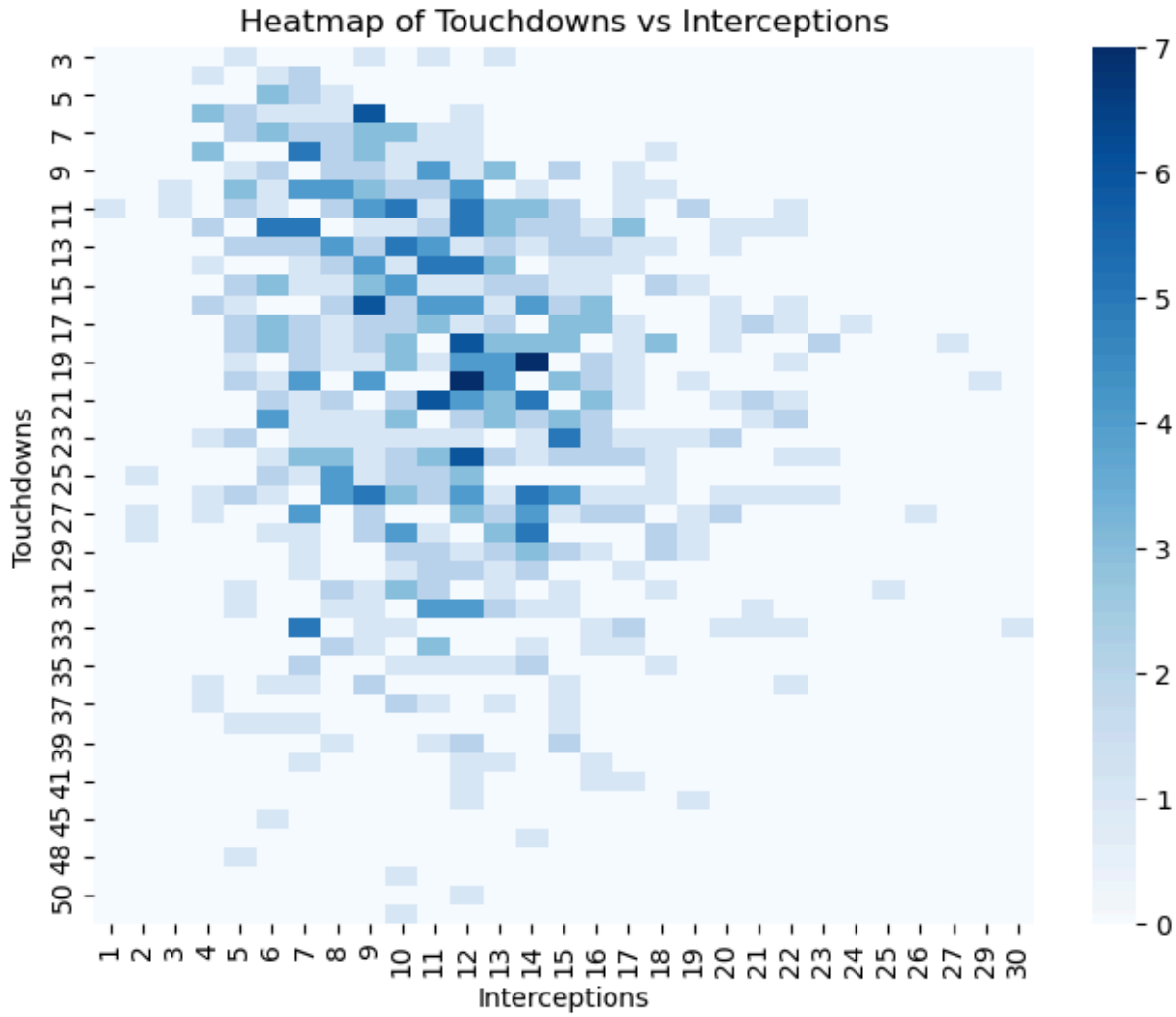
In [70]:
```
era_counts = pd.crosstab(starter_df["Era"], starter_df["Tm"])

era_counts.head()
# Helps examine how quarterback representation is distributed across teams a
```

Out[70]:

Tm Era	2TM	ARI	ATL	BAL	BUF	CAR	CHI	CIN	CLE	DAL	...	OAK	PHI	PIT	S
Early 2000s	1	10	9	10	11	9	8	10	9	8	...	8	10	9	
2010–2016	3	7	7	7	7	7	6	7	7	6	...	7	8	7	
Modern Era	3	6	7	7	7	6	9	8	6	7	...	3	7	7	

3 rows × 36 columns

In [71]:
```
# Based on the comments of my previous project, I have added an improved vis
import seaborn as sns
import matplotlib.pyplot as plt

td_int_table = pd.crosstab(starter_df["TD"], starter_df["Int"])

plt.figure(figsize=(8,6))
sns.heatmap(td_int_table, cmap="Blues")
plt.title("Heatmap of Touchdowns vs Interceptions")
plt.xlabel("Interceptions")
plt.ylabel("Touchdowns")
plt.show()
```



Project Assignment 6

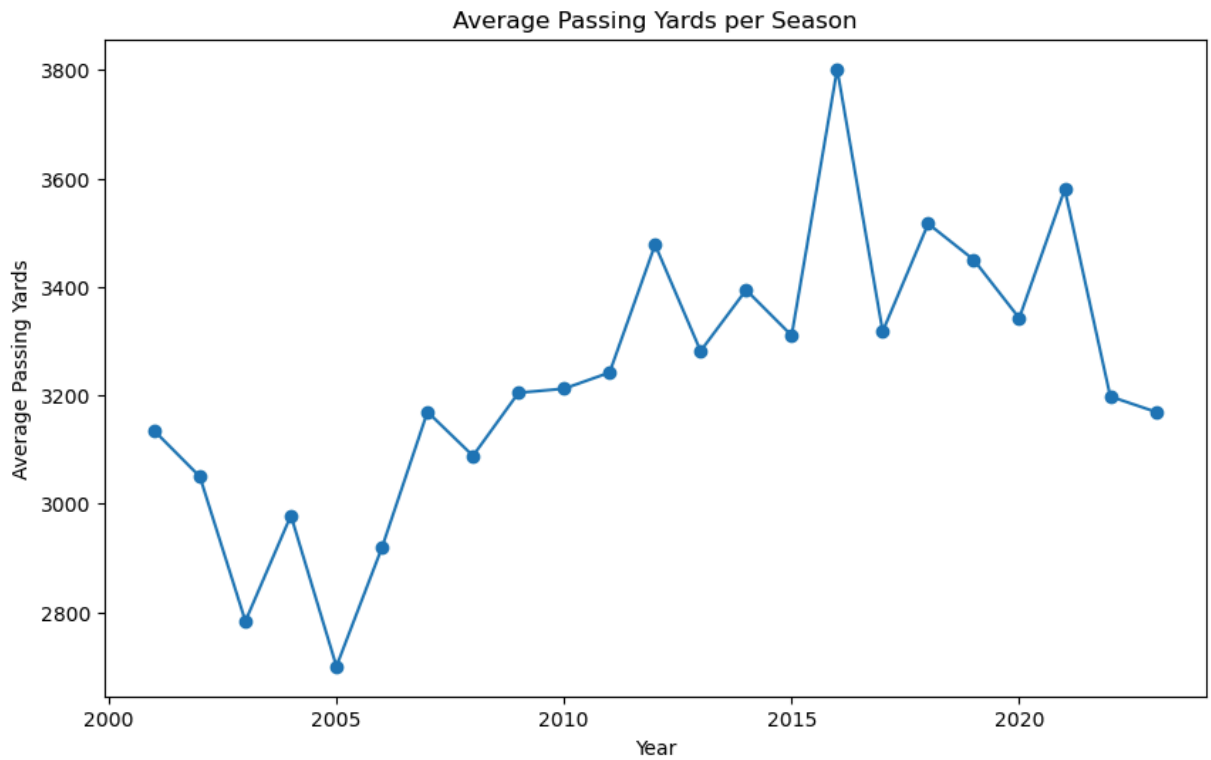
In [72]:
```
# League Avg Each Season
avg_by_year = (
    starter_df
    .groupby("Year")[["Yds","Rate","TD","Int"]]
    .mean()
    .reset_index()
)

avg_by_year.head()
```

Out[72]:

	Year	Yds	Rate	TD	Int
0	2001	3134.290323	79.548387	18.064516	14.935484
1	2002	3049.870968	81.706452	18.451613	12.774194
2	2003	2783.272727	78.569697	16.727273	13.272727
3	2004	2978.235294	84.473529	19.235294	12.411765
4	2005	2699.914286	81.445714	16.057143	11.257143

In [73]:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plt.plot(avg_by_year["Year"], avg_by_year["Yds"], marker="o")

plt.title("Average Passing Yards per Season")
plt.xlabel("Year")
plt.ylabel("Average Passing Yards")

plt.show()
```



In [74]:

```
# Average passing yards have generally increased over time.
# The NFL has gradually shifted toward pass-heavy offenses.
# Several rule changes have also contributed to this increase.
```

In [75]:

```
import statsmodels.api as sm

X = starter_df[["Yds","TD","Int","Att"]]
y = starter_df["Rate"]
```

```
X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
==
Dep. Variable:                   Rate   R-squared:                       0.8
94
Model:                            OLS   Adj. R-squared:                  0.8
94
Method:                 Least Squares   F-statistic:                     158
9.
Date:                Fri, 20 Mar 2026   Prob (F-statistic):               0.
00
Time:                        13:15:37   Log-Likelihood:                -210
2.2
No. Observations:                 757   AIC:                             421
4.
Df Residuals:                     752   BIC:                             423
8.
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
==
                 coef    std err          t      P>|t|      [0.025      0.97
5]
------------------------------------------------------------------------------
--
const         81.5718      0.586    139.211      0.000      80.422      82.7
22
Yds            0.0146      0.001     24.314      0.000       0.013       0.0
16
TD             0.6874      0.035     19.771      0.000       0.619       0.7
56
Int           -1.1287      0.037    -30.317      0.000      -1.202      -1.0
56
Att           -0.0939      0.004    -23.719      0.000      -0.102      -0.0
86
==============================================================================
==
Omnibus:                       36.499   Durbin-Watson:                   1.4
08
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               87.7
86
Skew:                          -0.230   Prob(JB):                     8.66e-
20
Kurtosis:                       4.604   Cond. No.                     1.41e+
04
==============================================================================
==

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
[2] The condition number is large, 1.41e+04. This might indicate that there
are
strong multicollinearity or other numerical problems.
```

In [76]:
```
# helps identify which statistics most influence quarterback performance.
```

In [77]:
```
era_avg = (
    starter_df
    .groupby("Era")[["Rate","Cmp%","Yds"]]
    .mean()
)

era_avg
```

```
/var/folders/_k/vn7_nw850bn6cd7c6b4273dm0000gn/T/ipykernel_86372/3327623.py:
3: FutureWarning: The default of observed=False is deprecated and will be ch
anged to True in a future version of pandas. Pass observed=False to retain c
urrent behavior or observed=True to adopt the future default and silence thi
s warning.
  .groupby("Era")[["Rate","Cmp%","Yds"]]
```

Out[77]:

Era	Rate	Cmp%	Yds
Early 2000s	82.785085	60.590847	2999.816949
2010–2016	87.606897	61.765517	3381.823276
Modern Era	91.635217	64.575217	3366.213043

In [78]:
```
# Key Findings:
# Passing yards have increased over time, reflecting the NFL's shift toward
# Regression results suggest touchdowns and interceptions are the strongest
# The touchdown-interception heatmap shows that elite seasons with high touc
```

In [ ]: